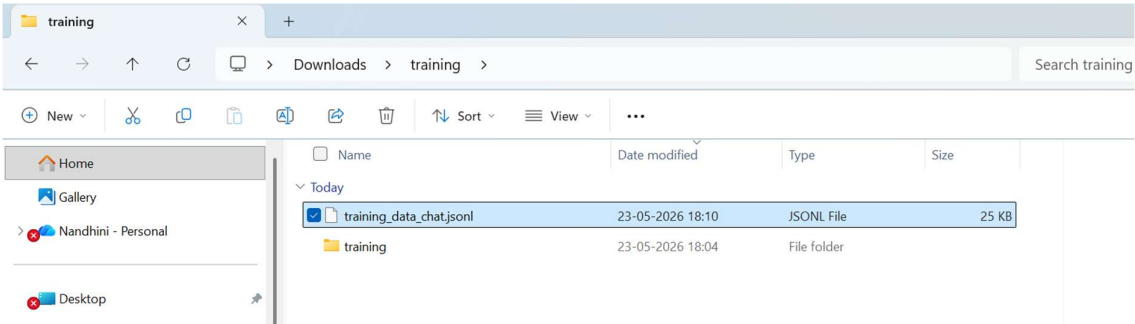


Training data .jsonl has been generated:



Uploaded the jsonl file in fine tuning and successfully completed the batch

Fine-tuning

Train and manage custom models

1
TOTAL JOBS

1
SUCCEEDED

0
FAILED

21,316
TRAINED TOKENS

AllSuccessfulFailed

+ Create

FINE-TUNE ID	BASE MODEL	METHOD	STATUS	TRAINED TOKENS	CREATED	
ft:gpt-4o-mini-2024-0...	gpt-4o-mini-2024-07...	supervised	Succeeded	21,316	23 May 2026 18:16	

1-1 of 1

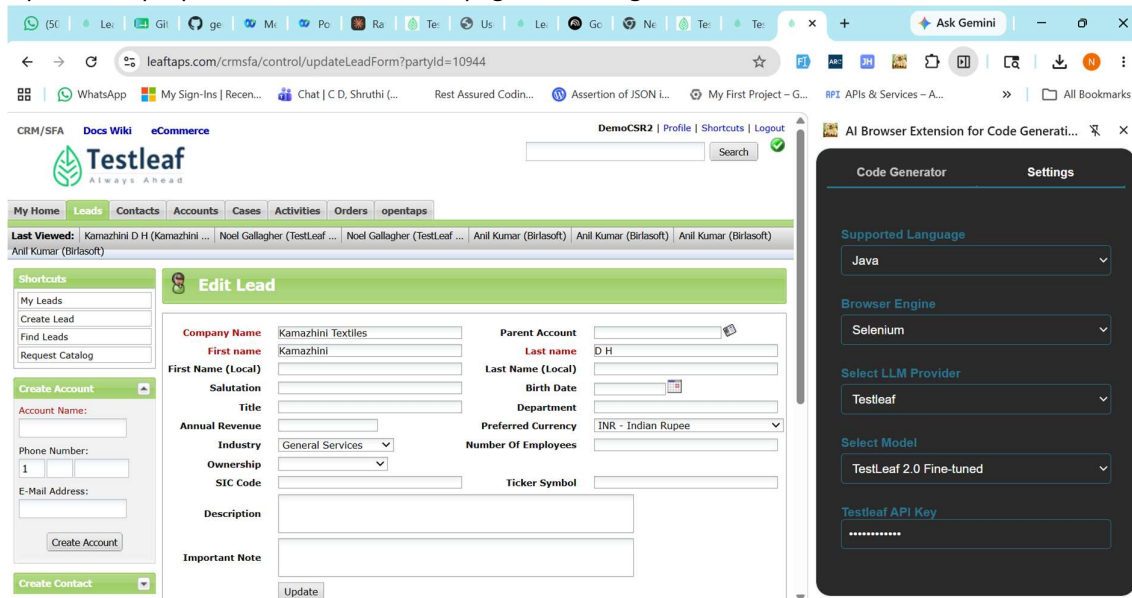
1

10 / page

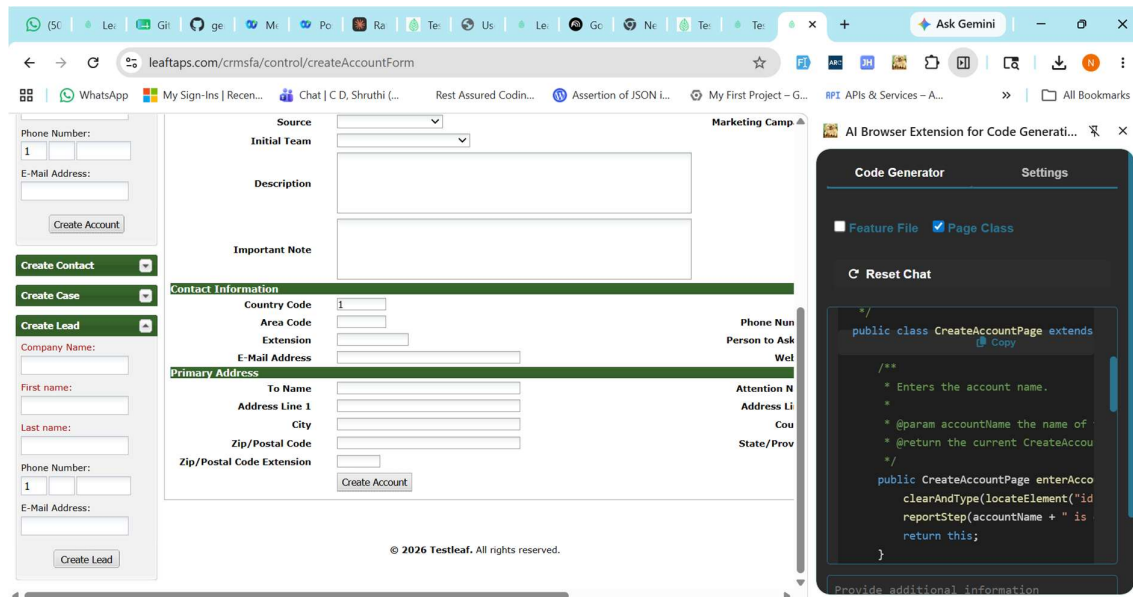
Updated the popup.json file in the ai_extension code base:

```
JS popup.js x Agenda.md Extension: Extension Pack for Java
src > scripts > JS popup.js > document.addEventListener('DOMContentLoaded') callback > testleaf > value
5 document.addEventListener('DOMContentLoaded', async () => {
104 const testleafApiKeyInput = document.getElementById('testleafApiKey');
105
106 // Model options by provider
107 const modelsByProvider = {
108   groq: [
109     { value: 'deepseek-r1-distill-llama-70b', label: 'deepseek-r1-distill-llama-70b' },
110     { value: 'llama-3.3-70b-versatile', label: 'llama-3.3-70b-versatile' },
111
112   ],
113   value: 'openai/gpt-oss-120b', label: 'openai/gpt-oss-120b' },
114   value: 'openai/gpt-oss-20b', label: 'openai-20b' },
115
116   ],
117   openai: [
118     { value: 'gpt-4o', label: 'GPT-4o' },
119     { value: 'gpt-4o-mini', label: 'GPT-4o Mini' },
120
121   ],
122   testleaf: [
123     { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DiZlHRx7', label: 'TestLeaf 1.0' },
124     { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::Dig7JbIP', label: 'TestLeaf 2.0 Fine-tuned' }
125   ]
126 };
127
128 // Function to update model options based on selected provider
129 function updateModelOptions(provider) {
130   modelSelect.innerHTML = '<option value="" disabled selected>Select the model...</option>';
131 }
```

Opened LeapTaps website and created a page class using the fine tuned model:



Now the extension is using the fine tuned model and generates the page class using the custom methods trained:



Page class code generated:
package com.testleaf.pages;

import org.openqa.selenium.By;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.ui.ExpectedConditions;

import org.openqa.selenium.support.ui.WebDriverWait;

/**

* Page Object for creating a new account.

*/

public class CreateAccountPage extends ProjectSpecificMethods {

/**

* Enters the account name.

*

* @param accountName the name of the account

* @return the current CreateAccountPage instance

*/

public CreateAccountPage enterAccountName(String accountName) {

```

clearAndType(locateElement("id", "accountName"), accountName);
reportStep(accountName + " is entered in Account Name field", "pass");
return this;
}

```

```

/**
 * Enters the group name.
 *
 * @param groupName the name of the group
 * @return the current CreateAccountPage instance
 */

```

```

public CreateAccountPage enterGroupName(String groupName) {
    clearAndType(locateElement("id", "groupNameLocal"), groupName);
    reportStep(groupName + " is entered in Group Name field", "pass");
    return this;
}

```

```

/**
 * Enters the annual revenue.
 *
 * @param annualRevenue the annual revenue amount
 * @return the current CreateAccountPage instance
 */

```

```

public CreateAccountPage enterAnnualRevenue(String annualRevenue) {
    clearAndType(locateElement("id", "annualRevenue"), annualRevenue);
    reportStep(annualRevenue + " is entered in Annual Revenue field", "pass");
    return this;
}

```

```

/**
 * Clicks the Create Account button.

```

```

*

* @return the MyAccountsPage instance
*/

public MyAccountsPage clickCreateAccountButton() {

    click(locateElement("xpath", "//input[@value='Create Account']"));

    reportStep("Create Account button is clicked", "pass");

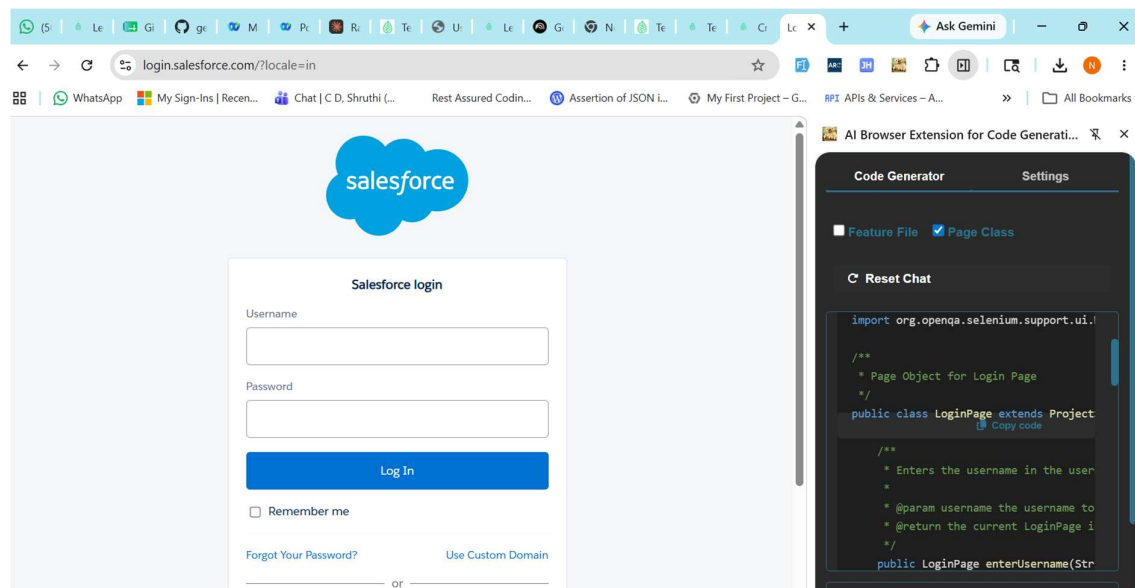
    return new MyAccountsPage();

}

}

```

Tried with a different website:



```
package com.testleaf.pages;
```

```

import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

/**
 * Page Object for Login Page

```

```
*/
```

```
public class LoginPage extends ProjectSpecificMethods {
```

```
/**
```

```
 * Enters the username in the username field.
```

```
 *
```

```
 * @param username the username to enter
```

```
 * @return the current LoginPage instance
```

```
*/
```

```
public LoginPage enterUsername(String username) {
```

```
    clearAndType(locateElement("id", "username"), username);
```

```
    reportStep(username + " is entered in the username field", "pass");
```

```
    return this;
```

```
}
```

```
/**
```

```
 * Enters the password in the password field.
```

```
 *
```

```
 * @param password the password to enter
```

```
 * @return the current LoginPage instance
```

```
*/
```

```
public LoginPage enterPassword(String password) {
```

```
    clearAndType(locateElement("id", "password"), password);
```

```
    reportStep(password + " is entered in the password field", "pass");
```

```
    return this;
```

```
}
```

```
/**
```

```
 * Clicks the login button to submit the form.
```

```
 *
```

```
 * @return the current HomePage instance
```

```

*/

public HomePage clickLoginButton() {
    click(locateElement("id", "Login"));
    reportStep("Login button is clicked", "pass");
    return new HomePage();
}

/**
 * Logs in with the specified username and password.
 *
 * @param username the username to enter
 * @param password the password to enter
 * @return the current HomePage instance
 */
public HomePage login(String username, String password) {
    enterUsername(username);
    enterPassword(password);
    return clickLoginButton();
}

/**
 * Waits for the login page to load.
 *
 * @return the current LoginPage instance
 */
public LoginPage waitForLoginPageToLoad() {
    new WebDriverWait(driver,
30).until(ExpectedConditions.visibilityOfElementLocated(By.id("username")));
    reportStep("Login page is loaded", "pass");
    return this;
}

```

